

SalePilot: Constraint-First Grounded Decision Support for Vietnamese Retail Consultation

1st Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

2nd Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

Abstract—Vietnamese retail consultation must preserve requirements across turns and never recommend items that violate budgets or physical limits. SalePilot grounds this task in fail-closed constraint filtering with verifiable decision evidence.

The system separates a deterministic constraint-first recommender from optional large language model (LLM) orchestration. Each recommendation carries constraint states, admission counts, provenance, and a catalog hash for rechecking.

We evaluate on a sealed benchmark of 40 Vietnamese episodes over a hash-pinned snapshot of 13,716 products in 118 categories, comparing the hybrid constraint ranker with lexical-filter and price-popularity baselines.

Explicit constraint filtering removes every observed violation: 0 in 99 product-constraint checks for both constraint conditions versus 10 in 117 for the price-popularity baseline. The paired bootstrap difference of 0.0855 excludes zero.

All conditions share one parser and tie on the remaining metrics; ranking differences need relevance labels, scoped to future work. Deployment claims stay prototype-scoped. Artifacts are hash-chained.

Index Terms—conversational recommender systems, decision support, Vietnamese language, constraint satisfaction, explainable recommendation, reproducibility

I. INTRODUCTION

Conversational recommender systems (CRSs) turn one-shot ranking into an interaction that can elicit preferences, explain options, and react to feedback [1].

For appliances and electronics, a short request may omit budget, room or household size, physical limits, or energy preferences. The system must decide whether to clarify, recommend, answer a policy question, or abstain.

Selective questioning has been formalized for recommendation [2]. Estimation–action–reflection separates preference estimation, action choice, and feedback handling [3].

Recent work uses LLMs as planners over recommender tools [4]. Hybrid multi-agent designs have also been studied for e-commerce consultation [5].

Fluency alone is insufficient. CRS evaluation must distinguish component quality from user experience [6]. Agent benchmarks likewise motivate multi-turn tests in executable environments [7].

SalePilot asks three questions. Does explicit filtering reduce observed hard-constraint violations? Does its hybrid ranker outperform a lexical-filter baseline? Which system and deployment claims do the current artifacts support?

The paper makes three contributions:

- 1) A Vietnamese prototype that combines structured need accumulation, deterministic ranking, optional LLM orchestration, and a multi-backend catalog;
- 2) A versioned evidence dictionary for deterministic recommendations, including selected product facts, computed constraint states, fail-closed admission counts, and catalog identity metadata;
- 3) A sealed, manifest-linked evaluation of three deterministic conditions on one catalog snapshot, with paired bootstrap inference, explicit custody, and evidence limits.

The experiment does not test the Lead loop, specialist roles, response fluency, interface quality, conversion, or field effectiveness. These boundaries prevent component-level results from becoming system-wide claims.

II. RELATED WORK AND POSITIONING

A. Conversational preference state

CRS research spans intent handling, knowledge, interaction, and evaluation [1]. The System Ask–User Respond paradigm explicitly elicits attributes [8].

EAR separates preference estimation, asking or recommending, and reflection after feedback [3]. SalePilot adopts this decomposition as inspectable rules and state, not as a reproduction of either learned model.

B. Tool-grounded and role-scoped agents

ReAct interleaves model reasoning with actions and observations [9]. Multi-agent work organizes systems through profiles, communication, and skills [10].

AutoGen provides a programmable precedent for multi-agent conversation [11]. SalePilot instead keeps product recommendation on a deterministic path when a request is sufficiently clear.

External stores motivate retrieval-augmented generation [12] and retrieval-in-the-loop conversation [13]. SalePilot retrieves catalog records, but no result from those methods transfers to this system.

C. Explanation, evaluation, and reproducibility

Explainable recommendation serves different stakeholders and goals [14]. A supported catalog claim is not proof that text faithfully exposes a model’s causal process [15].

Recommendation metrics measure different properties [16]. For example, nDCG combines graded relevance with rank discount [17]; this study lacks relevance labels and does not report nDCG.

Simple baselines can expose weak progress claims [18]. Offline protocols can also contain methodological flaws [19].

We therefore separate action, extraction, constraint, and deployment evidence.

III. SALEPILOT DESIGN

A. System boundary and orchestration

Fig. 1 shows the boundary: a runtime path from channels to ranking, and the deterministic decision data that every emitted recommendation carries.

Clear product requests enter a deterministic fast path for category detection, need merge, clarification, and ranking. With no LLM key, a rule-based coordinator also handles comparison, FAQ, CRM, and escalation branches.

With an LLM key, the same fast path is attempted first. Ambiguous requests may enter a Lead-tool loop. Catalog and knowledge are direct Lead tools; short tool-scoped loops exist for named specialist roles.

The evaluated crawl registry has hand-authored rules for eight core families. Other categories loaded from the catalog receive generic objects with aliases and budget-oriented ranking, but no category-specific need slots.

A separate workbook profile declares 14 categories for that data configuration. The sealed experiment instead uses the crawl profile and a 118-category DMX snapshot.

B. Need state and action policy

Let $H_t = (u_1, \dots, u_t)$ be the conversation and n_t a structured need dictionary. A category-aware extractor E and merge operator M update the state:

$$n_t = M(n_{t-1}, E(u_t, c_{t-1})). \quad (1)$$

Fresh nonempty values overwrite matching fields. Priorities are unioned within one category. A category switch drops earlier category fields but retains the old budget when the new turn gives none.

On the keyed fast path, a bare number or recognized room label may fill the first pending slot. The offline coordinator does not apply this resolver, so the two runtime paths are not behaviorally identical.

Runtime routing uses flags rather than a typed action enum. The benchmark allows `recommend`, `clarify`, `compare`, `faq`, and `abstain`; the current adapter emits all except `compare`.

A missing budget triggers clarification unless flexibility is explicit. A core category can also require one primary slot.

Generic categories usually require only category and budget before ranking.

Fig. 2 contrasts a one-shot reply with this clarify-then-recommend policy; the questions shown are a clarification exchange recorded from the running prototype.

In sealed episode test-001, *Gia đình 3 người cần tủ lạnh dưới 12 triệu inverter* parses into the category, a household size, a budget, and an energy-saving priority in the recorded need.

C. Constraint-first ranking

The ranker considers only products with a current price in the selected category. A nonflexible over-budget item is rejected; a flexible over-budget item is retained with a penalty.

Configured range, minimum, maximum, and proximity terms compare normalized product facts with the need. Missing hard facts reject an item unless the slot policy explicitly permits unknown values.

At a high level, the score groups budget, slot, preference, and secondary terms:

$$S(p, n_t) = B(p, n_t) + \sum_k w_k F_k(p, n_t) + P(p, n_t) + \epsilon(p). \quad (2)$$

The implementation also uses brand, style, rating, popularity, discount, and gift signals. Equation (2) is a grouping of code paths, not a claim that the evidence payload exposes every addend.

Products are ordered by descending score and ascending price. Equal ties inherit repository order, so repeatability depends on fixed input order rather than a complete cross-backend tie key.

A second pass removes exact model-code duplicates. It favors distinct brands among the first two selections, then fills up to three products. It does not perform similarity-based near-duplicate detection.

D. Decision evidence and catalog identity

Deterministic recommendation routes emit a versioned evidence dictionary. It contains the parsed need, selected SKUs, total match scores, computed constraint statuses, selected product facts, and source metadata.

Constraint states are `matched`, `violated`, or `unknown`. Provenance can include source row, active backend, and a semantic catalog digest; source-row values can be absent.

The dictionary also counts the priced candidates evaluated and those rejected fail-closed. Clarification turns leave both counts empty because no candidate set was examined.

The payload supports rechecking listed constraints against catalog facts. It is not a faithful explanation of LLM reasoning, an independent certificate, or a complete reconstruction of the ranking score.

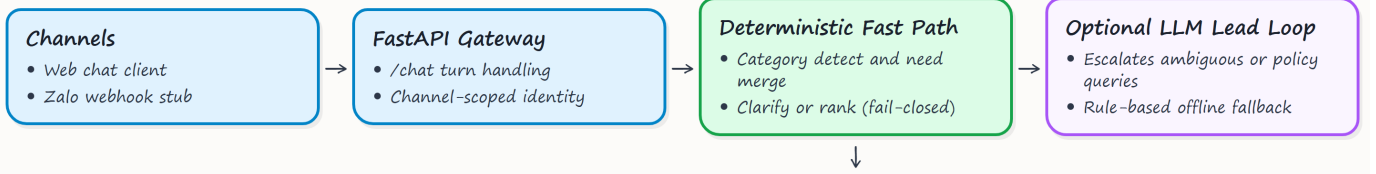
PostgreSQL-first and MongoDB-first modes both end at a JSON snapshot; snapshot mode is isolated. Fallback reports the backend loaded but does not prove that different backends contain identical records.

Research scripts separately validate and pin snapshot bytes. Runtime loading does not consult that manifest or verify

SalePilot: Constraint-First Decision-Support Architecture

Request flow: runtime path hands off to the deterministic decision-data plane

1. RUNTIME PATH



2. DETERMINISTIC DECISION DATA

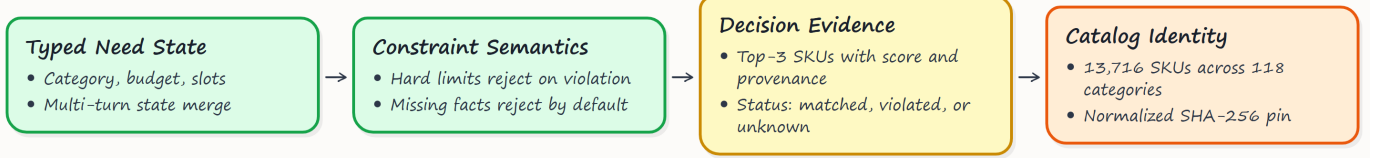


Fig. 1. SalePilot architecture. The runtime path (top) separates the deterministic fast path from the optional LLM Lead loop; the decision data plane (bottom) turns typed need state and fail-closed constraint semantics into evidence tied to a hash-pinned catalog identity.

One-Shot Reply vs. Clarify-Then-Recommend

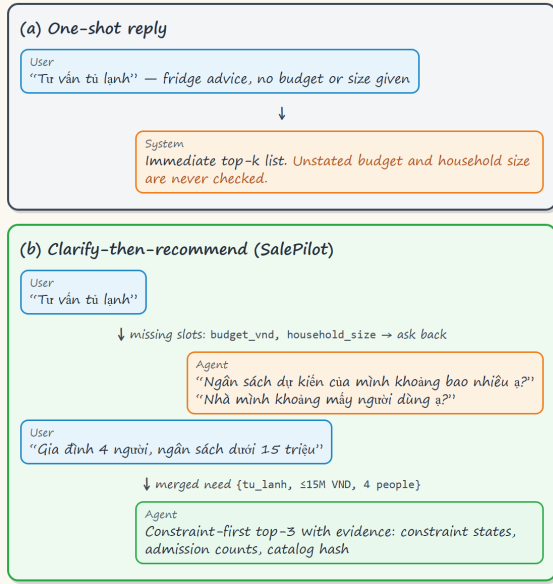


Fig. 2. One-shot reply versus clarify-then-recommend. With required slots missing, the agent asks back and merges the answers into the need state before constraint-first ranking; the dialogue is recorded from the prototype.

its declared counts; it computes a semantic digest over the documents that actually load.

E. Operational and deployment scope

The prototype keys state by caller-supplied channel and external identity. The web client stores an unsigned local UUID. No server-signed web session or per-identity turn lock is implemented.

Runtime trajectory capture stores raw identifiers and conversational content when enabled. The research release policy

instead forbids raw trajectories and real contact details, reflecting data-minimization goals [20].

An optional command tool restricts command names and its working directory, but its path checks are partial. The fetch tool blocks literal local addresses, yet lacks host allowlisting and redirect-target validation.

Indirect prompt injection motivates stronger server-side boundaries for retrieved content [21]. The current controls reduce some exposure but should not be described as a secure jail or an SSRF-resistant boundary.

The repository defines a five-service Docker Compose prototype: MongoDB, PostgreSQL, PostgREST, FastAPI, and Next.js. The advisor can rank without an LLM key, while the full API still uses configured persistence services.

Each request is tagged with its serving route: offline coordinator, deterministic fast path, LLM loop, or LLM-error fallback. The API exposes per-route counts with p50 and p95 latency for deployment monitoring.

IV. EVALUATION PROTOCOL

A. Benchmark and experimental boundary

Fig. 3 summarizes the executable pipeline: every condition consumes the same sealed episodes and hash-pinned catalog, and every output artifact is hashed for independent reruns.

The sealed test set contains 40 developer-authored, single-turn Vietnamese episodes. It covers 11 non-null product categories plus FAQ and out-of-scope cases, including clarification, abstention, negation, and no-diacritics inputs.

The fixed DMX snapshot contains 13,716 products in 118 categories. Its normalized SHA-256 is shown below, with one layout line break:

```
e2afdf38ed2df5599611e6f8285643f7
27e2c06c721b1a055b7baeea797df15c
```

Sealed Evaluation Pipeline

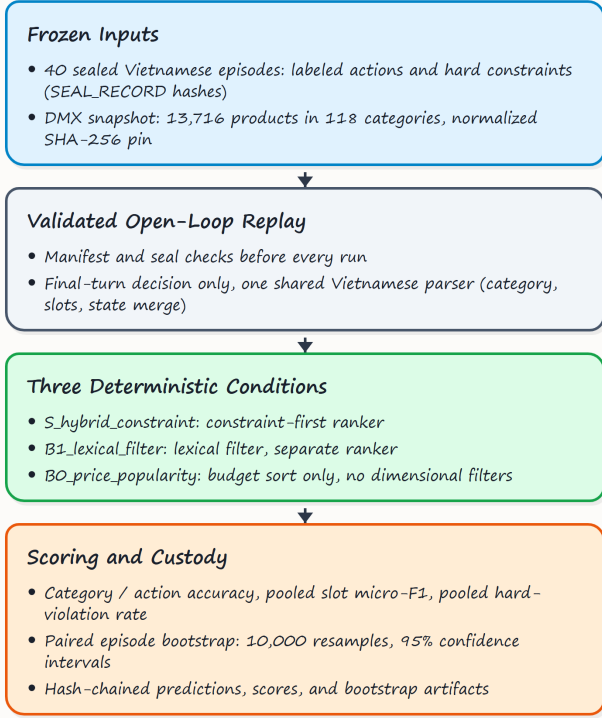


Fig. 3. Sealed evaluation pipeline. Identical frozen inputs and one shared parser isolate action policy and ranking as the only varying factors.

Turns are replayed open-loop. Only the prediction after the final user turn is scored. The Lead loop, specialist roles, databases, persistence, generated wording, frontend, and channel controls are not executed.

B. Systems

All three systems share category detection, slot extraction, and state merge. This isolates action and ranking differences but prevents the comparison from testing alternative Vietnamese parsers.

S_hybrid_constraint uses the production ranker with configured slot and preference terms. *B1_lexical_filter* uses lexical overlap, popularity, explicit filters, and a separate ranking implementation.

B0_price_popularity filters missing prices and over-budget items, then sorts by popularity and price. It does not apply the remaining dimensional or category-specific filters.

An optional LLM baseline is omitted from the primary comparison. Its checked-in provider pin, prediction bytes, catalog, and score artifacts do not form one consistent custody chain.

C. Metrics and custody

Category accuracy includes all 40 episodes, including null-category labels. Action accuracy covers the four classes emitted by these systems: recommend, clarify, FAQ, and abstain.

Slot micro-F1 pools true positives, false positives, and false negatives across structured need fields. It is not the mean of episode-level F1 values.

Hard-constraint violation rate pools product–constraint pairs over returned products. A check compares one returned product with one labeled hard constraint, applying that constraint’s missing-data policy.

Prediction, score, and bootstrap files are hashed in a chain-of-custody record. The test file and catalog are also hash-pinned, although the record does not pin a code revision or conditions-file hash.

Frozen inputs and explicit configurations address documented reproducibility problems in recommender research [22].

The artifact practice also mirrors parts of the NeurIPS reproducibility program, including code submission and checklists [23].

V. RESULTS AND EVIDENCE BOUNDARY

A. Quantitative results

Table I reports the three deterministic conditions on the same sealed split and DMX snapshot.

TABLE I
RESULTS ON 40 SEALED VIETNAMESE EPISODES AND THE 13,716-PRODUCT DMX SNAPSHOT

System	Cat.	Act.	Slot μ F1	Hard Viol.	Chk ^a
<i>S_hybrid</i>	0.950	0.850	0.942	0.000	99
<i>B1_lexical</i>	0.950	0.850	0.942	0.000	99
<i>B0_popularity</i>	0.950	0.925	0.942	0.0855	117

^aCat./Act.: category and action accuracy. Slot μ F1: pooled slot micro-F1. Hard Viol.: pooled violations per product–constraint check. Chk: checks evaluated.

All conditions obtain 0.950 category accuracy and 0.942 slot micro-F1 because they share the same parser. These ties do not show that the parser is optimal or that ranking has no effect.

B0 obtains higher action accuracy: 0.925 versus 0.850. Its simpler admission policy avoids some clarifications, but its returned lists produce 10 violations across 117 checks.

Those violations comprise six area checks, three budget checks, and one width check. B1 and *S_hybrid* produce no observed violations across 99 checks each.

The exposure also differs: B0 faces 117 product–constraint checks versus 99 because B1 and *S_hybrid* more often clarify instead of recommending, so violation removal combines filtering with a more conservative action policy.

Episode-resample bootstrap intervals (10,000 resamples) estimate the pooled table metrics directly. All conditions are evaluated on identical resamples, so violation-rate differences are paired.

B0’s pooled violation rate is 0.0855 with 95% CI [0.010, 0.167]. The paired B0 minus *S_hybrid* difference is 0.0855 with the same interval, excluding zero; the B0 minus B1 difference is identical.

B1 and S_hybrid show zero violations in all resamples, so their percentile intervals are degenerate at zero. Under a rule-of-three reading that treats checks as independent, 0 of 99 bounds the rate below about 0.03 [24].

B1 and S_hybrid tie on every reported aggregate. Their top-three SKU sequences differ on 28 episodes, but the benchmark has no item-relevance labels. It cannot establish which ranking is more useful.

The evidence supports one statistically grounded claim: explicit constraint filtering eliminates the observed violations that the price–popularity policy admits. It does not show superiority of S_hybrid over B1 or generalization.

B. Decision-evidence boundary

The S_hybrid artifacts preserve selected facts and computed statuses for returned products. A reviewer can re-run the evaluator against the pinned snapshot without invoking an LLM or network service.

The same implementation constructs the status fields and recommendations. The payload is therefore inspectable evidence, not independent certification. Frontend rendering and human comprehension were not evaluated.

C. Deployment evidence

At the recorded public check on 25 July 2026, the frontend returned HTTP 200 in 426.96 ms. The backend health endpoint returned HTTP 502 in 138.2 ms.

Chat, product, and fallback probes were skipped after the failed health check. The evidence supports public frontend reachability and local prototype configuration, not a healthy public end-to-end deployment.

D. Threats to validity

The benchmark is developer-authored, not an independently sampled or dual-annotated field corpus. Rule, scenario, and evaluator co-development can overfit the 40 episodes.

Open-loop final-turn replay does not test interactive question policy or user adaptation. Shared extraction makes the comparison chiefly about action and filtering, not end-to-end CRS quality.

The catalog covers 118 categories, but the test has 11 labeled product categories. No relevance judgments support ranking metrics, and the inconsistent optional LLM artifacts prevent a like-for-like B2 comparison.

Reported intervals resample whole episodes and recompute pooled metrics [25]. They capture episode-sampling variability only, not authorship bias or the dependence of multiple checks inside one episode.

The custody record pins data and output hashes but not the code revision or conditions hash. Runtime identity, privacy, tool safety, orchestration, and public availability also remain outside the experiment.

Our planned external study targets at least 80 Vietnamese conversations, two independent annotators, and graded relevance labels. This sample target is project-specific and requires a separate precision analysis.

We will choose agreement coefficients according to label scale and assumptions [26]. Uncertainty analysis will resample whole conversations and specify the bootstrap interval [25].

User-facing constructs will be declared from CRS-Que before collection [27]. Until that study is complete, we make no claim about usefulness, trust, fluency, or conversion.

VI. CONCLUSION

SalePilot is a Vietnamese retail prototype that separates deterministic recommendation from optional LLM orchestration and emits inspectable evidence for its deterministic routes.

On one sealed, developer-authored 40-episode benchmark, B1 and S_hybrid show no observed violations in 99 checks, while B0 shows 10 in 117; the paired difference of 0.0855 (95% CI 0.010–0.167) excludes zero.

These results support explicit fail-closed filtering within the evaluated setup. They do not establish ranking utility over B1, end-to-end agent quality, field effectiveness, or healthy public deployment.

The artifact hashes, evidence dictionary, and candid scope boundary provide a basis for independent reruns. External annotation, relevance judgments, and an auditable LLM baseline remain future gates.

ACKNOWLEDGMENT

AI coding assistants supported language drafting, bibliographic collection, diagram review, and consistency checks against source code and evaluator artifacts. They did not generate the reported measurements.

The authors remain responsible for verifying the manuscript, sources, measurements, and disclosure before submission.

REFERENCES

- [1] D. Jannach, A. Manzoor, W. Cai, and L. Chen, “A survey on conversational recommender systems,” *ACM Computing Surveys*, vol. 54, no. 5, pp. 105:1–105:36, 2021. [Online]. Available: <https://doi.org/10.1145/3453154>
- [2] K. Christakopoulou, F. Radlinski, and K. Hofmann, “Towards conversational recommender systems,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 2016, pp. 815–824. [Online]. Available: <https://doi.org/10.1145/2939672.2939746>
- [3] W. Lei, X. He, Y. Miao, Q. Wu, R. Hong, M.-Y. Kan, and T.-S. Chua, “Estimation–action–reflection: Towards deep interaction between conversational and recommender systems,” in *Proceedings of the 13th ACM International Conference on Web Search and Data Mining*. Association for Computing Machinery, 2020, pp. 304–312. [Online]. Available: <https://doi.org/10.1145/3336191.3371769>
- [4] X. Huang, J. Lian, Y. Lei, J. Yao, D. Lian, and X. Xie, “Recommender AI agent: Integrating large language models for interactive recommendations,” *ACM Transactions on Information Systems*, vol. 43, no. 4, pp. 96:1–96:33, 2025. [Online]. Available: <https://doi.org/10.1145/3731446>
- [5] G. Nie, R. Zhi, X. Yan, Y. Du, X. Zhang, J. Chen, M. Zhou, H. Chen, T. Li, Z. Cheng, S. Xu, and J. Hu, “A hybrid multi-agent conversational recommender system with LLM and search engine in e-commerce,” in *Proceedings of the 18th ACM Conference on Recommender Systems*. Association for Computing Machinery, 2024, pp. 745–747. [Online]. Available: <https://doi.org/10.1145/3640457.3688061>
- [6] D. Jannach, “Evaluating conversational recommender systems: A landscape of research,” *Artificial Intelligence Review*, vol. 56, no. 3, pp. 2365–2400, 2023. [Online]. Available: <https://doi.org/10.1007/s10462-022-10229-x>

- [7] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *The Twelfth International Conference on Learning Representations*. OpenReview.net, 2024. [Online]. Available: <https://openreview.net/forum?id=zAdUB0aCTQ>
- [8] Y. Zhang, X. Chen, Q. Ai, L. Yang, and W. B. Croft, "Towards conversational search and recommendation: System ask, user respond," in *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. Association for Computing Machinery, 2018, pp. 177–186. [Online]. Available: <https://doi.org/10.1145/3269206.3271776>
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *The Eleventh International Conference on Learning Representations*. OpenReview.net, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [10] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*. International Joint Conferences on Artificial Intelligence Organization, 2024, pp. 8048–8057, Survey Track. [Online]. Available: <https://www.ijcai.org/proceedings/2024/890>
- [11] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversations," in *Proceedings of the First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=BAakY1hNKS>
- [12] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. F. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>
- [13] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval augmentation reduces hallucination in conversation," in *Findings of the Association for Computational Linguistics: EMNLP 2021*. Association for Computational Linguistics, Nov. 2021, pp. 3784–3803. [Online]. Available: <https://aclanthology.org/2021.findings-emnlp.320/>
- [14] Y. Zhang and X. Chen, "Explainable recommendation: A survey and new perspectives," *Foundations and Trends in Information Retrieval*, vol. 14, no. 1, pp. 1–101, 2020. [Online]. Available: <https://doi.org/10.1561/15000000066>
- [15] A. Jacovi and Y. Goldberg, "Towards faithfully interpretable NLP systems: How should we define and evaluate faithfulness?" in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2020, pp. 4198–4205. [Online]. Available: <https://aclanthology.org/2020.acl-main.386/>
- [16] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, "Evaluating collaborative filtering recommender systems," *ACM Transactions on Information Systems*, vol. 22, no. 1, pp. 5–53, 2004. [Online]. Available: <https://doi.org/10.1145/963770.963772>
- [17] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of IR techniques," *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002. [Online]. Available: <https://doi.org/10.1145/582415.582418>
- [18] M. Ferrari Dacrema, S. Boglio, P. Cremonesi, and D. Jannach, "A troubling analysis of reproducibility and progress in recommender systems research," *ACM Transactions on Information Systems*, vol. 39, no. 2, pp. 20:1–20:49, 2021. [Online]. Available: <https://doi.org/10.1145/3434185>
- [19] B. Hidasi and Á. T. Czapp, "Widespread flaws in offline evaluation of recommender systems," in *Proceedings of the 17th ACM Conference on Recommender Systems*. Association for Computing Machinery, 2023, pp. 848–855. [Online]. Available: <https://doi.org/10.1145/3604915.3608839>
- [20] National Institute of Standards and Technology, "NIST privacy framework: A tool for improving privacy through enterprise risk management, version 1.0," National Institute of Standards and Technology, Gaithersburg, MD, USA, NIST Cybersecurity White Paper NIST CSWP 10, Jan. 2020. [Online]. Available: <https://doi.org/10.6028/NIST.CSWP.10>
- [21] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, ser. AISec '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 79–90. [Online]. Available: <https://doi.org/10.1145/3605764.3623985>
- [22] J. Beel, C. Breitingner, S. Langer, A. Lommatzsch, and B. Gipp, "Towards reproducibility in recommender-systems research," *User Modeling and User-Adapted Interaction*, vol. 26, no. 1, pp. 69–101, 2016. [Online]. Available: <https://doi.org/10.1007/s11257-016-9174-x>
- [23] J. Pineau, P. Vincent-Lamarre, K. Sinha, V. Larivière, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and H. Larochelle, "Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021. [Online]. Available: <https://www.jmlr.org/papers/v22/20-303.html>
- [24] J. A. Hanley and A. Lippman-Hand, "If nothing goes wrong, is everything all right? Interpreting zero numerators," *JAMA*, vol. 249, no. 13, pp. 1743–1745, 1983. [Online]. Available: <https://doi.org/10.1001/jama.1983.03330370053031>
- [25] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979. [Online]. Available: <https://doi.org/10.1214/aos/1176344552>
- [26] R. Artstein and M. Poesio, "Survey article: Inter-coder agreement for computational linguistics," *Computational Linguistics*, vol. 34, no. 4, pp. 555–596, 2008. [Online]. Available: <https://doi.org/10.1162/coli.07-034-R2>
- [27] Y. Jin, L. Chen, W. Cai, and X. Zhao, "CRS-Que: A user-centric evaluation framework for conversational recommender systems," *ACM Transactions on Recommender Systems*, vol. 2, no. 1, pp. 2:1–2:34, 2024. [Online]. Available: <https://doi.org/10.1145/3631534>